

SUPER PROMPT — App de Rifa (Role + XML + Chain of Thought)

Pega este bloque completo en Claude Code. Está diseñado para que Claude razone paso a paso antes de escribir código y entregue una app funcional, en línea y en tiempo real.

<rol> Eres un desarrollador full-stack senior especializado en aplicaciones web en tiempo real, con experiencia en React, bases de datos, control de concurrencia y despliegue. Escribes código limpio, seguro y listo para producción, y explicas tus decisiones de arquitectura de forma breve. Piensas primero en la estructura de datos y en los casos límite antes de escribir una sola línea. </rol> <contexto> Necesito una aplicación web para gestionar una **rifa**: - **10,000 números** en total, numerados del **0000** al **9999** (formato de 4 dígitos con ceros a la izquierda). - Cada número cuesta **5 dólares (USD)**. Recaudación máxima potencial: **\$50,000**. - Dos administradoras: **Thaidis** y **Daniela**. - La app debe estar **en línea**, ser **interactiva** y mostrar el **avance en tiempo real** a todos los que entren, desde cualquier dispositivo (celular o computadora). - Los compradores no pagan dentro de la app; el pago se coordina aparte. La app registra la selección del número, genera un ticket y las administradoras confirman el pago. </contexto> <objetivo> Construir una aplicación web con **dos vistas accesibles por link**: 1. **Vista pública** (link para compartir con todos): cualquiera entra, ve la cuadrícula de números, selecciona uno disponible, ingresa sus datos, recibe un ticket y ve el avance de la rifa en vivo. 2. **Vista de administración** (link protegido para Thaidis y Daniela): panel con métricas, porcentaje vendido, lista de compradores y acciones de gestión. </objetivo> <requisitos_funcionales>

<vista_publica>

- Cuadrícula (o buscador + cuadrícula) con los 10,000 números, mostrando visualmente el estado de cada uno:
 - **Disponible** (se puede seleccionar).
 - **Reservado** (alguien lo tomó, pago pendiente — no seleccionable).
 - **Pagado / Vendido** (no seleccionable).
- Al seleccionar un número disponible, el usuario ingresa: **nombre completo, teléfono (WhatsApp) y correo (opcional)**.
- Al confirmar, el número pasa a estado **Reservado**, se guarda en la base de datos y se genera un **ticket** con: número comprado, nombre del comprador, un **código único de ticket**, fecha/hora y monto (\$5). El ticket debe poder verse en pantalla y descargarse/imprimirse (o guardarse como imagen/PDF).

- **Barra o indicador de avance en tiempo real** visible para el público: cuántos números se han tomado y el porcentaje, actualizándose solo cuando otros compran (sin recargar la página).
- Un mismo número **no puede ser seleccionado por dos personas a la vez** (control de concurrencia atómico: si dos personas intentan el mismo número al mismo tiempo, solo uno lo consigue y el otro recibe un aviso).
- Buscador para escribir un número específico y ver si está disponible.
- Diseño responsivo, limpio y fácil de usar en el celular. </vista_publica>

<vista_administracion>

- Acceso protegido por inicio de sesión solo para **Thaidis** y **Daniela**.
- **Dashboard** con:
 - **Porcentaje vendido** de la rifa (pagados / 10,000) mostrado de forma prominente (número grande + barra de progreso).
 - Cantidad de números **disponibles, reservados y pagados**.
 - **Monto recaudado** (pagados × \$5) y **monto potencial pendiente**.
- **Tabla de participantes** con: número, nombre, teléfono, correo, estado, código de ticket y fecha. Con filtros (por estado) y buscador.
- **Acciones administrativas:**
 - Marcar un número reservado como **Pagado**.
 - **Liberar** un número (volver a disponible) por si alguien se arrepiente o no paga.
 - Registrar o editar manualmente un comprador (para ventas hechas en efectivo/presenciales).
- **Exportar** la lista completa a CSV/Excel.
- Todo se actualiza también en tiempo real. </vista_administracion>

<general> - Estado compartido y persistente: si alguien compra, ****todos**** los demás (público y administradoras) ven el cambio al instante, sin recargar. - Los datos deben ****persistir**** aunque se cierre el navegador o se reinicie (base de datos real, no memoria local). </general>

</requisitos_funcionales>

<especificaciones_tecnicas>

- **Recomendación de stack:** React (con Vite o Next.js) para el frontend y **Supabase** (base de datos Postgres + suscripciones en tiempo real + autenticación) para el backend compartido. Supabase resuelve de forma sencilla la persistencia, el tiempo real y el login de administradoras, y tiene capa gratuita. Si consideras que otra opción

es mejor (por ejemplo Firebase o un backend Node/Express con WebSockets), propónmela justificándola brevemente antes de proceder.

- **Modelo de datos sugerido (ajústalo si mejora):**

- Tabla `numeros`: `numero` (0000-9999, clave), `estado` ('disponible' | 'reservado' | 'pagado'), `comprador_nombre`, `comprador_telefono`, `comprador_correo`, `codigo_ticket`, `fecha`.
- Tabla `administradores` para el login (o usar el sistema de auth del stack elegido).

- **Concurrencia:** la reserva de un número debe ser una operación atómica (por ejemplo, un UPDATE condicional que solo tenga éxito si el número sigue 'disponible'), para evitar ventas duplicadas.
- **Tiempo real:** usa suscripciones/realtime para que la cuadrícula y las métricas se actualicen automáticamente en todos los clientes.
- **Seguridad básica:** las acciones de administración (marcar pagado, liberar, exportar, editar) solo deben ser accesibles tras iniciar sesión. La vista pública nunca debe exponer la capacidad de modificar estados administrativos.
- **Despliegue:** entrégame instrucciones claras y paso a paso para poner la app en línea gratis (por ejemplo Vercel o Netlify para el frontend y Supabase para la base de datos), incluyendo dónde colocar las claves/variables de entorno y cómo crear las cuentas de las dos administradoras. </especificaciones_tecnicas>

<estados_de_numero>

- **disponible:** color neutro, seleccionable.
- **reservado:** color de advertencia (ej. amarillo/naranja), no seleccionable, cuenta pago pendiente.
- **pagado:** color de éxito (ej. verde), no seleccionable, cuenta como venta confirmada. El **porcentaje vendido** de la rifa se calcula con los **pagados**. La barra pública puede mostrar reservados + pagados como "avance total". </estados_de_numero>

<flujo_de_razonamiento> Antes de escribir código, razona paso a paso y muéstrame tu plan:

1. Confirma el stack que usarás y por qué encaja con "tiempo real para todos" y "persistente".
2. Define el esquema exacto de la base de datos y los estados.
3. Explica cómo garantizarás que dos personas no compren el mismo número simultáneamente.
4. Describe cómo fluye el tiempo real desde que alguien reserva hasta que todos ven el cambio.
5. Lista las pantallas/rutas: pública, ticket, login admin, dashboard admin.

6. Solo después de esto, genera el código completo, organizado por archivos, y al final las instrucciones de despliegue y de creación de las cuentas de Thaidis y Daniela. Si algo es ambiguo, elige la opción más sencilla y robusta, dímelo, y continúa sin detenerte.
- </flujo_de_razonamiento>

< criterios_de_aceptacion>

- Puedo compartir un link público y varias personas pueden entrar a la vez y ver el mismo avance actualizándose en vivo.
- Al seleccionar un número y llenar los datos, se genera un ticket con código único y el número queda reservado para todos.
- Dos personas no pueden quedarse con el mismo número.
- Thaidis y Daniela entran por un link con login, ven el **porcentaje vendido**, el monto recaudado y la lista de compradores, y pueden marcar pagos, liberar números y exportar.
- Los datos siguen ahí si cierro y vuelvo a abrir.
- Tengo instrucciones claras para dejarlo en línea gratis. </ criterios_de_aceptacion>

< entregables> 1. El código completo del proyecto, organizado por archivos. 2. El script/esquema de la base de datos. 3. Un README con: cómo instalar, cómo configurar variables de entorno, cómo crear las cuentas de las dos administradoras y cómo desplegar en línea paso a paso. 4. Los dos links (o rutas) claramente identificados: público y administración. </ entregables>